TRABAJO PRÁCTICO · SUB-PROYECTO 3

Estilización del HTML con CSS

Caso de estudio: Nutrite — Dietética Online

Materia	Aplicaciones Web I — Sistemas
Profesor	Andrés Senn
Alumno	Franco Tellechea
Fecha	Octubre de 2026 — Córdoba
Dominio propuesto	www.nutrite.com.ar

Contenido

1. Introducción y alcance
2. 3.1 Creación de los estilos
3. 3.2 Nomenclatura de clases (BEM)
4. 3.3 Responsividad (media queries)
5. 3.4 Organización de recursos en directorios
6. 3.5 Cambios respecto de la propuesta
7. Anexo — Uso de Inteligencia Artificial

1. Introducción y alcance

En el trabajo anterior se definió la propuesta UX/UI de **Nutrite**: ficha de negocio, proto-persona (Julieta Fernández, 29 años, usuaria mobile-first de Instagram), dos historias de usuario, seis requerimientos funcionales de frontend, el sitemap y la estrategia MPA/SPA. En este sub-proyecto se llevó esa propuesta a un sitio maquetado en **HTML5 semántico** y estilizado completamente con **CSS3**, sin frameworks (ni Bootstrap ni Tailwind) y sin JavaScript.

Páginas desarrolladas

Página	Archivo	Sección del sitemap	Requerimientos que cubre
Inicio	`index.html`	Home: hero, categorías, destacados, propuesta de valor, Instagram	R1 (buscador), R4 (carrito en header)
Catálogo	`pages/catalogo.html`	Catálogo / Tienda con filtros y atributos	R1 (filtros por atributo y categoría), R2 (stock en cada tarjeta)
Detalle de producto	`pages/producto.html`	Imágenes, descripción, atributos, stock, precio y cuotas	R2, R3, R5 (WhatsApp), R6 (cuotas)
Carrito	`pages/carrito.html`	Resumen, calculadora de envío, acceso a checkout	R3, R4 (editar cantidad / eliminar)
Checkout	`pages/checkout.html`	Checkout por pasos (stepper), paso 3: método de pago	R6 (medios de pago y cuotas antes de confirmar)
Contacto	`pages/contacto.html`	Formulario, WhatsApp/Instagram, FAQ	R5

Cómo ver el proyecto

Descomprimir la carpeta `nutrite/` y abrir `index.html` en cualquier navegador moderno. Todas las rutas son relativas, por lo que funciona sin servidor. Para probar la responsividad se puede achicar la ventana o usar las herramientas de desarrollador (modo dispositivo).

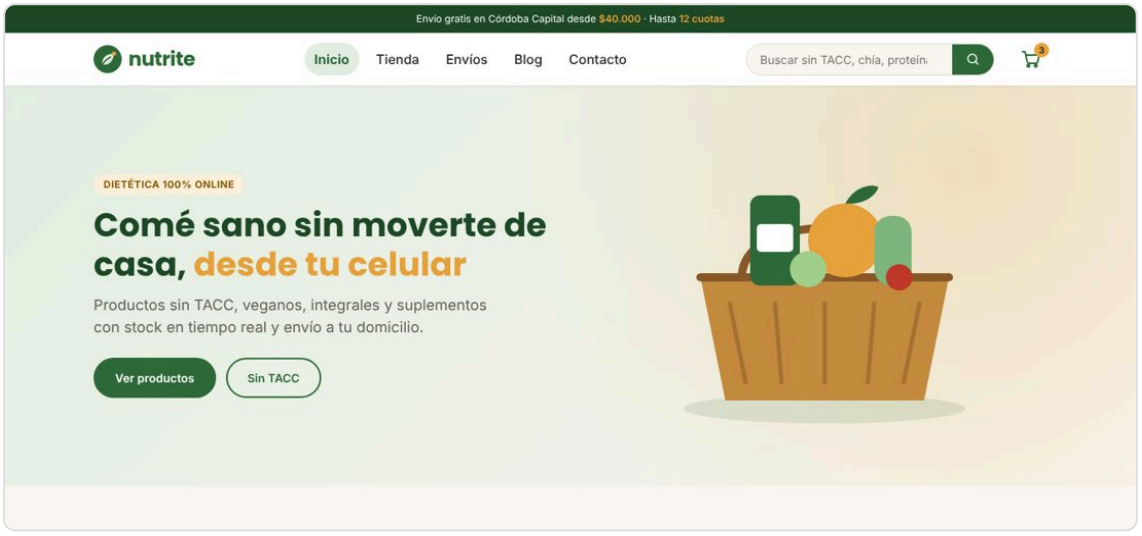


Figura 1 — Página de inicio en escritorio (1366 px).

3.1 Creación de los estilos

Los estilos se construyeron a partir de la identidad que ya tiene el negocio en Instagram (natural, cercana, saludable) y de la proto-persona: Julieta compra desde el celular, revisa rápido y desconfía cuando no ve stock o costo de envío. Por eso la interfaz prioriza **legibilidad, botones grandes y táctiles, y estados bien diferenciados por color**.

Variables CSS (design tokens)

Todos los valores visuales se centralizaron como custom properties en `base/variables.css`. Así, cambiar un color de marca o un espaciado se hace en un solo lugar y se propaga a todo el sitio.

--color-primario
#2f6b3a verde hoja

--color-primario-oscuro
#1f4a28

--color-acento
#e8a33d miel

--color-fondo
#faf7f0 crema

--color-stock-ok
En stock

--color-stock-bajo
Últimas unidades

--color-stock-sin
Sin stock

```
:root {
  --color-primario: #2f6b3a;     --fuente-titulos: "Poppins", system-ui, sans-serif;
  --color-acento:   #e8a33d;     --fuente-texto:   "Inter", system-ui, sans-serif;
  --esp-1: 0.25rem; ... --esp-8: 4rem;     /* escala de espaciado de 4px */
  --radio-md: 12px;  --sombra-sm: 0 1px 3px rgba(43,43,43,.08);
  --ancho-max: 1200px;  --transicion: 0.2s ease;
}
```

Tipografía

Poppins (600/700) para títulos y precios —redondeada y amigable— e **Inter** (400–700) para textos, por su gran legibilidad en pantallas chicas. Se cargan desde Google Fonts con `font-display: swap` y tienen `system-ui` como respaldo. Se usan unidades `rem` para respetar el tamaño de fuente que configure el usuario.

Componentes visuales y su relación con los requerimientos

Componente (bloque CSS)	Qué resuelve	Req.
`.buscador`, `.filtros`, `.chip`, `.badge`	Búsqueda, filtros por atributo/categoría, chips de filtros activos y etiquetas de color (Sin TACC, Vegano, Sin azúcar, Integral) visibles en cada tarjeta.	R1 · HU1
`.stock` con modificadores `--disponible`, `--ultimas`, `--sin-stock`	Indicador semáforo con punto de color generado con `::before`. En la ficha, la variante `--grande` con animación de pulso para "últimas unidades". Las tarjetas sin stock se ven en escala de grises y con el botón deshabilitado.	R2 · HU2
`.calculadora-envio`, `.opcion-envio`	Ingreso de código postal y opciones con costo y tiempo estimado, en la ficha de producto y en el carrito (antes de pagar). La opción elegida se resalta con `:has(input:checked)`.	R3 · HU2

`.carrito-icono`, `.item-carrito`, `.cantidad`	Ícono con contador presente en el header de todas las páginas; ítems editables (+/−, eliminar) y subtotal.	R4
`.btn--whatsapp`, `.whatsapp-flotante`	Botón directo en la ficha de producto y botón flotante fijo en todo el sitio.	R5
`.stepper`, `.medio-pago`, `.cuotas`	Pasos del checkout con contador CSS (`counter()`), tarjetas de medios de pago y tabla de cuotas visible antes de confirmar.	R6

Técnicas CSS aplicadas

- **CSS Grid** para layouts de página (header, catálogo con sidebar, carrito + resumen, grillas de productos) y **Flexbox** para alineaciones internas de componentes.
- **Pseudo-elementos** (`::before` / `::after`) para el punto de stock, separadores del breadcrumb, líneas del stepper e íconos +/− de acordeones.
- **Pseudo-clases** `:hover`, `:focus-visible`, `:focus-within`, `:checked`, `:has()`, `:not()`, `:nth-child()`.
- **Transiciones y animaciones** (`transition`, `@keyframes`) en botones, tarjetas, menú hamburguesa y stock bajo.
- **Accesibilidad**: foco visible en color acento, clase `.u-visualmente-oculto` para etiquetas de lectores de pantalla, contraste suficiente entre texto y fondo, y `<details>/<summary>` nativos para acordeones.
- **Sin estilos en línea**: todo el diseño vive en las hojas de estilo; el HTML solo contiene estructura y clases.

3.2 Nomenclatura de clases: metodología BEM

Se adoptó **BEM (Bloque, Elemento, Modificador)** como criterio de nombres. Cada componente es un bloque independiente; sus partes internas son elementos y sus variantes o estados son modificadores. Esto vuelve explícita la jerarquía de cada clase con solo leer su nombre, evita choques de nombres entre componentes y mantiene la especificidad baja y pareja (casi todo se estiliza con una sola clase).

```
.bloque                  →  componente independiente     .producto-card
.bloque__elemento        →  parte del bloque              .producto-card__precio
.bloque--modificador     →  variante o estado del bloque  .producto-card--sin-stock
.bloque__elemento--mod   →  variante de un elemento       .nav__link--activo
```

Reglas que se siguieron

- Nombres en **español** y en **kebab-case** (minúsculas separadas por guiones): `calculadora-envio`, `item-carrito`.
- Un solo nivel de elemento: se escribe `.producto-card__nombre`, nunca `.producto-card__cuerpo__nombre`, aunque en el HTML el nombre esté dentro del cuerpo.
- Los estados se expresan como modificadores (`--activo`, `--completo`, `--deshabilitado`), no con clases sueltas.
- No se usan **IDs** para dar estilo (solo para vincular `label` con `input`) ni selectores de etiqueta dentro de componentes salvo excepciones puntuales.
- Además de BEM se usan dos **prefijos** para distinguir clases que no son componentes:
  - `l-` = *layout*: estructura general reutilizable (`.l-contenedor`, `.l-seccion`, `.l-grilla`).
  - `u-` = *utilidad*: clases de un único propósito (`.u-visualmente-oculto`, `.u-texto-centro`, `.u-mt-4`).

Ejemplo completo: tarjeta de producto

```
<article class="producto-card producto-card--sin-stock">
  <a class="producto-card__imagen-link"><img class="producto-card__imagen"></a>
  <div class="producto-card__cuerpo">
    <span class="producto-card__categoria">Suplementos</span>
    <h3 class="producto-card__nombre">Proteína vegana de arveja 1 kg</h3>
    <div class="badges"><span class="badge badge--vegano">Vegano</span></div>
    <span class="stock stock--sin-stock">Sin stock</span>
    <span class="producto-card__precio">$18.500</span>
    <button class="btn btn--chico btn--deshabilitado">Sin stock</button>
  </div>
</article>
```

Se ve cómo conviven bloques distintos (`producto-card`, `badge`, `stock`, `btn`) sin depender unos de otros: el mismo `.stock` se reutiliza en la ficha de producto y en el carrito, y el mismo `.btn` en todo el sitio.

Inventario de bloques

Tipo	Bloques
Layout (`l-`)	l-contenedor, l-seccion, l-grilla
Estructura del sitio	barra-promo, header, logo, nav, buscador, carrito-icono, breadcrumb, footer, whatsapp-flotante

Componentes	btn, badge / badges, producto-card, categoria-card, chip, campo, form, cantidad, stock, calculadora-envio, opcion-envio, acordeon, paginacion
De página	hero, beneficios, categorias, banner-instagram, catalogo, filtros, producto, tabla-nutricional, carrito, item-carrito, resumen, stepper, checkout, medio-pago, cuotas, contacto, canal, faq
Utilidades (`u-`)	u-visualmente-oculto, u-texto-centro, u-mt-4, u-mt-6, u-solo-mobile, u-solo-desktop

3.3 Responsividad

Se trabajó con enfoque **mobile-first**: los estilos base están pensados para el celular (el dispositivo principal de la proto-persona) y los **media queries con `min-width`** van agregando columnas y reacomodando elementos a medida que la pantalla crece. Además se incluye `<meta name="viewport" content="width=device-width, initial-scale=1.0">` en todas las páginas.

Breakpoints

Rango	Media query	Cambios principales
Celular < 600 px	*(estilos base)*	Menú hamburguesa; buscador en una fila completa debajo del logo; grilla de 2 productos; filtros plegables en un `<details>`; carrito y resumen apilados; footer en una columna; botón de WhatsApp solo con ícono.
Tablet chica	`@media (min-width: 600px)`	Buscador en la misma fila que el logo; grilla de 3 productos; ítems del carrito en una fila (imagen · info · acciones); formularios en 2 columnas; footer en 2 columnas.
Tablet	`@media (min-width: 768px)`	Hero en 2 columnas (texto + ilustración); grilla de 4 productos.
Escritorio	`@media (min-width: 1024px)`	Menú horizontal visible (desaparece la hamburguesa); catálogo con sidebar de filtros fija (`position: sticky`); carrito y checkout con resumen lateral fijo; ficha de producto en 2 columnas con galería sticky; títulos más grandes; footer en 4 columnas.
Escritorio XL	`@media (min-width: 1280px)`	Catálogo con 4 columnas de productos junto a la sidebar.

Técnicas que acompañan a los media queries

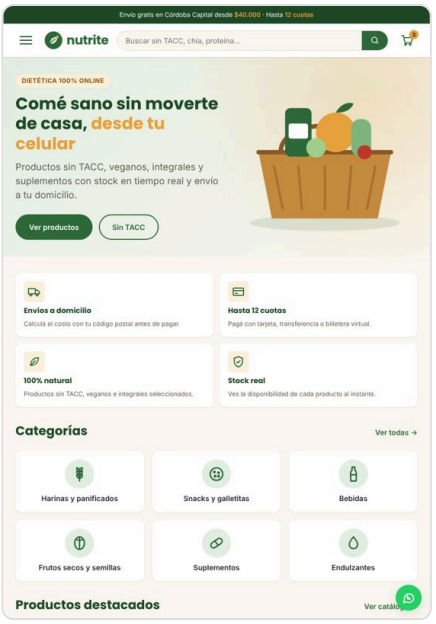
- `grid-template-areas` en el header: el mismo HTML se reorganiza en mobile (“toggle logo carrito / buscador”) y en escritorio (“logo nav buscador carrito”).
- **Menú hamburguesa sin JavaScript**: un `checkbox` oculto + `label`; con `:checked ~ .nav` se despliega el menú y las tres líneas se transforman en una “X”.
- `minmax(0, 1fr)` en las grillas de página para evitar desbordes horizontales en pantallas angostas (se verificó que ninguna página tenga scroll horizontal a 390 px).
- Imágenes fluidas (`max-width: 100%`, `aspect-ratio: 1/1`) y unidades relativas (`rem`, `%`, `fr`, `ch`).



Inicio — 390 px



Menú desplegado — 390 px

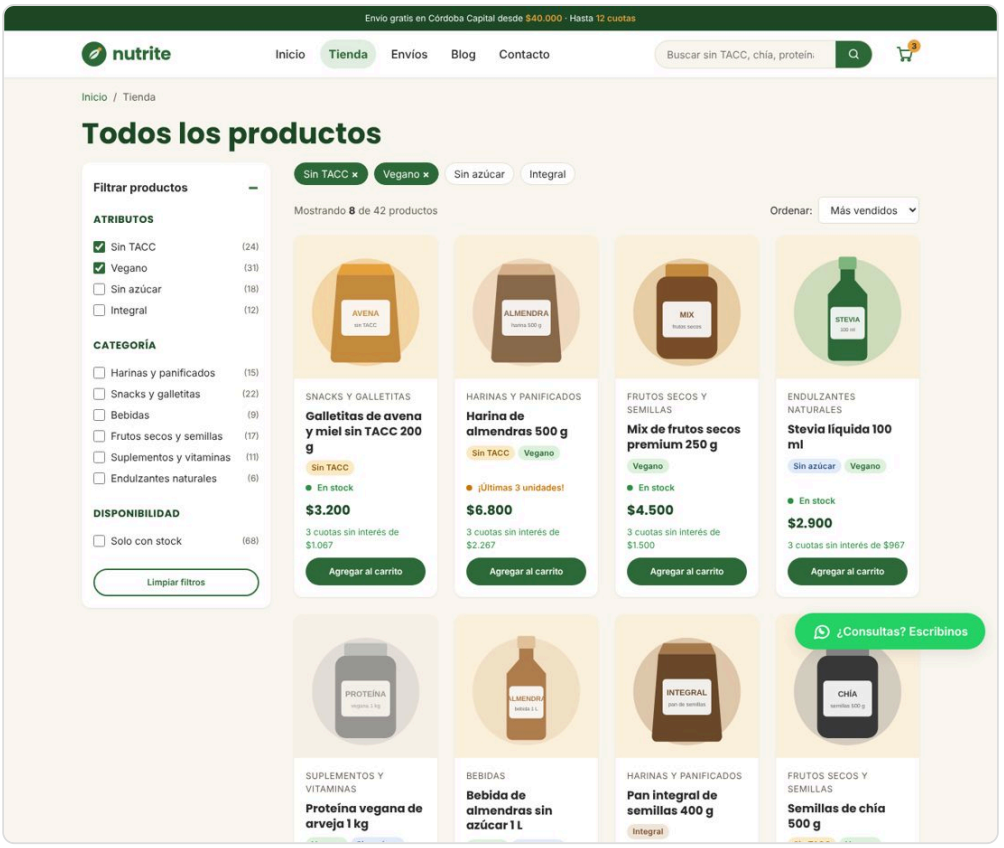


Inicio — tablet 820 px

Catálogo: misma página en celular y escritorio

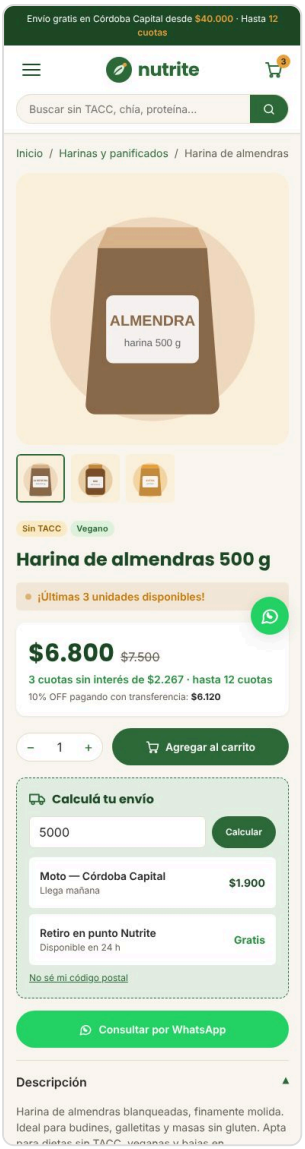


390 px: filtros plegables y 2 columnas

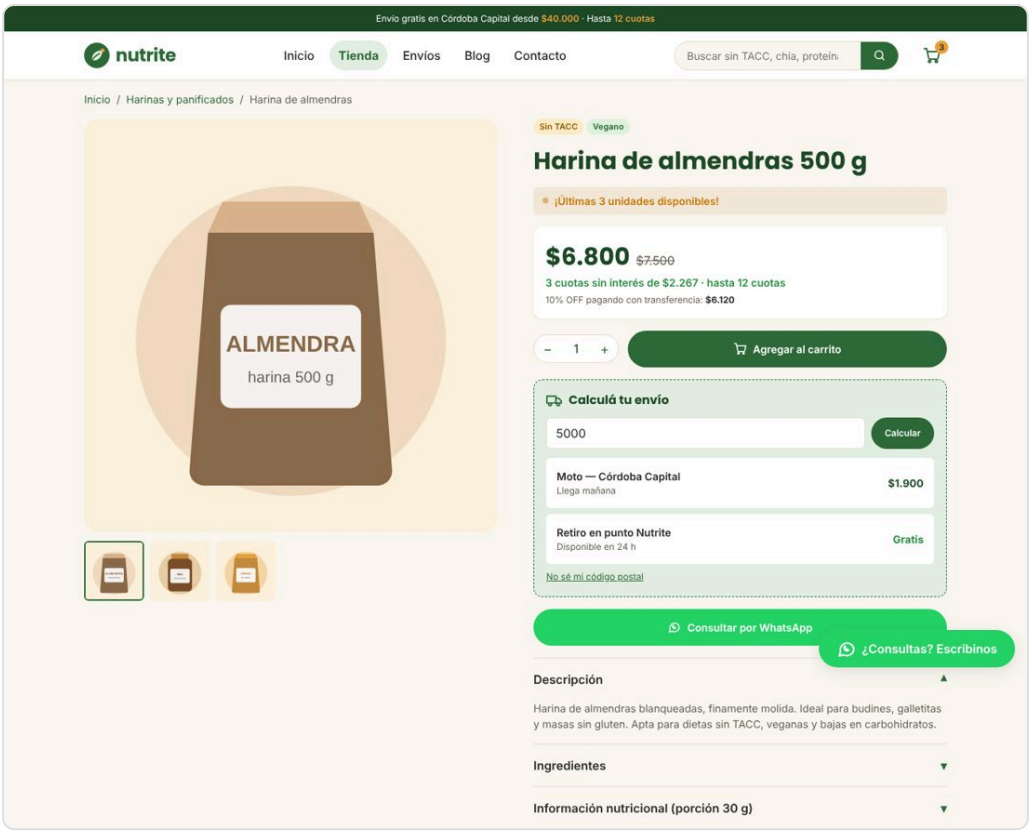


1366 px: sidebar sticky y 4 columnas

Ficha de producto

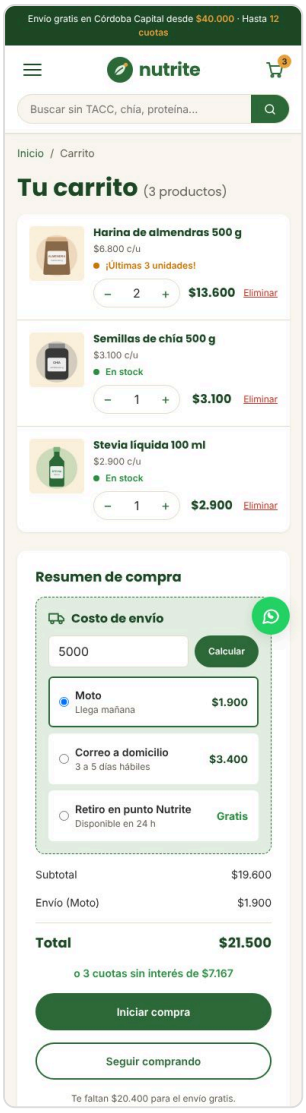


Ficha — 390 px

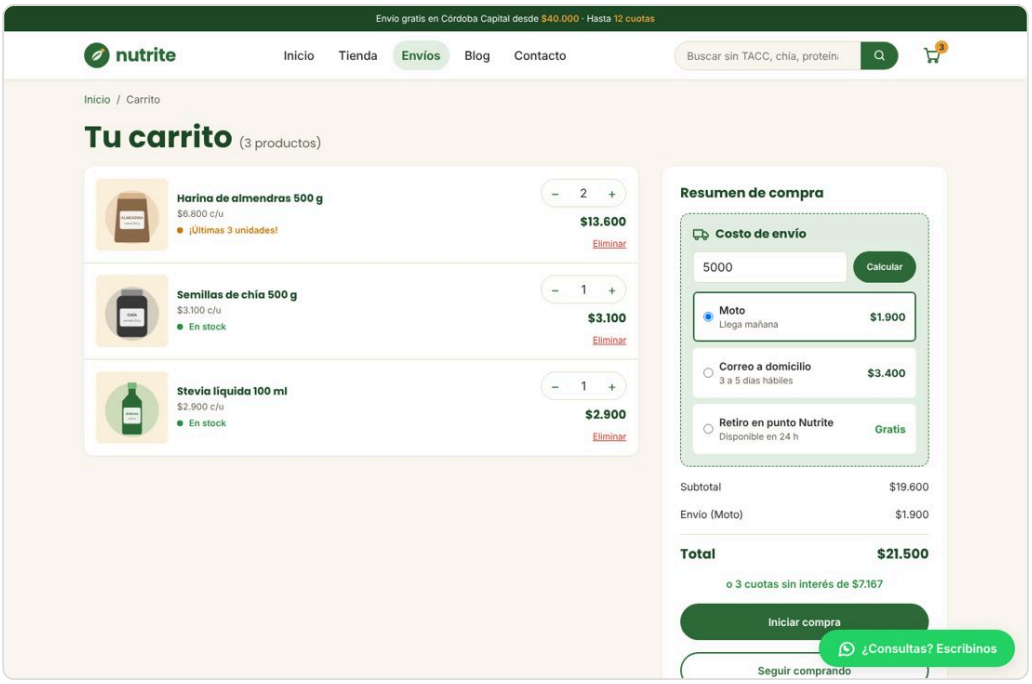


Ficha — 1366 px: stock, cuotas, calculadora de envío y WhatsApp

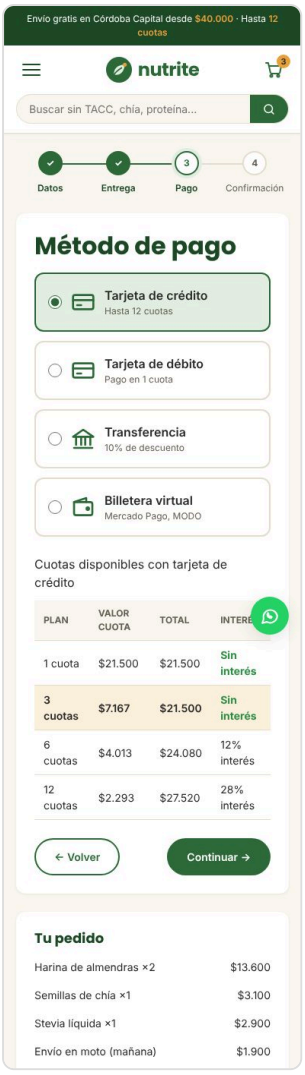
Carrito y checkout



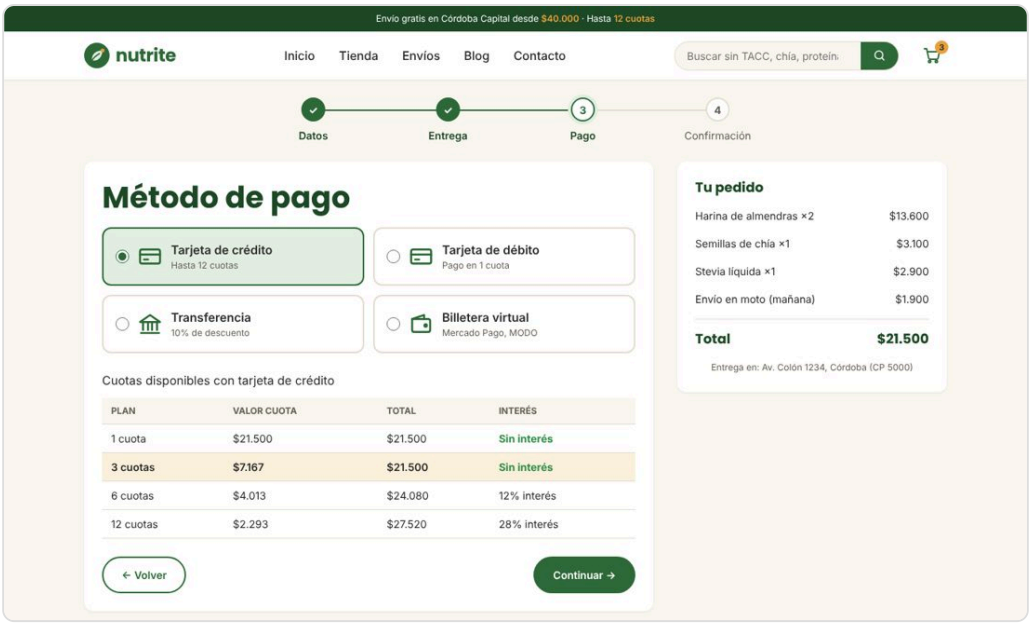
Carrito — 390 px



Carrito — 1366 px: resumen lateral con calculadora de envío



Checkout — 390 px



Checkout — 1366 px: stepper, medios de pago y cuotas

3.4 Organización de recursos en directorios

Los recursos se separaron por tipo y, dentro de los estilos, por responsabilidad. Las páginas internas viven en `pages/` y todos los recursos estáticos en `assets/`.

```
nutrite/
├── index.html                  ← página de inicio
├── pages/
│   ├── catalogo.html
│   ├── producto.html
│   ├── carrito.html
│   ├── checkout.html
│   └── contacto.html
└── assets/
    ├── css/
    │   ├── main.css            ← único archivo enlazado; importa al resto
    │   ├── base/               ← fundamentos globales
    │   │   ├── variables.css   (design tokens)
    │   │   ├── reset.css
    │   │   ├── tipografia.css
    │   │   └── utilidades.css  (clases u-)
    │   ├── layout/             ← estructura común a todas las páginas
    │   │   ├── layout.css      (clases l-, breadcrumb)
    │   │   ├── header.css
    │   │   └── footer.css
    │   ├── components/         ← piezas reutilizables
    │   │   ├── botones.css  ├── badges.css   ├── tarjetas.css
    │   │   ├── formularios.css ├── stock.css  └── envio.css
    │   └── pages/              ← estilos propios de cada página
    │       ├── inicio.css   ├── catalogo.css  ├── producto.css
    │       ├── carrito.css  ├── checkout.css  └── contacto.css
    └── img/
        ├── logo.svg
        ├── hero-canasta.svg
        ├── iconos/             (íconos SVG de interfaz y categorías)
        └── productos/          (imágenes de productos)
```

Orden de la cascada

Cada página enlaza solo `assets/css/main.css`, que importa los parciales con `@import` en orden de especificidad creciente: **base → layout → componentes → páginas → utilidades**. De esta manera los estilos generales se definen primero, los específicos los refinan, y las utilidades quedan al final para poder sobrescribir cuando se necesita. Cada archivo CSS tiene responsabilidad única y un comentario de cabecera, lo que facilita encontrar dónde se define cada cosa.

```
/* main.css */
@import url("base/variables.css");     @import url("components/botones.css");
@import url("base/reset.css");         @import url("components/stock.css");   ...
@import url("layout/header.css");      @import url("pages/catalogo.css");     ...
@import url("layout/footer.css");      @import url("base/utilidades.css");    /* al final */
```

> Las rutas son relativas: desde `index.html` se usa `assets/...` y desde las páginas internas `../assets/...`. En producción, los `@import` podrían unificarse en un solo archivo minificado para reducir peticiones HTTP; para la entrega se priorizó la legibilidad y la organización.

3.5 Cambios respecto de la propuesta

La implementación respetó la estructura del sitemap, la proto-persona, las historias de usuario y los seis requerimientos funcionales. Los cambios y agregados fueron los siguientes:

Alcance de esta etapa

En la propuesta	En lo desarrollado	Motivo
Sitemap completo (11 secciones)	Se maquetaron las 6 pantallas del flujo de compra: Inicio, Catálogo, Producto, Carrito, Checkout y Contacto/FAQ. Mi Cuenta, Blog, Información de envíos y Páginas legales quedan enlazadas desde el menú y el footer, pero no se maquetaron todavía.	Se priorizó el recorrido que resuelve HU1 y HU2. Las páginas restantes son mayormente texto y reutilizan los mismos componentes.
Módulos SPA: filtros en tiempo real, buscador predictivo, carrito dinámico, calculadora de envío y stepper	Se diseñaron todos sus **estados visuales** con CSS (filtro activo, chips, stock en 3 estados, opción de envío seleccionada, pasos completos/activos), con datos de ejemplo fijos.	El SP3 trata la estilización. La lógica (JavaScript) se incorpora en la etapa siguiente; las clases BEM ya están preparadas para que el script solo agregue o quite modificadores.
Checkout de 4 pasos	Se maquetó el paso 3 (Pago) con el stepper mostrando los 4 pasos y su progreso.	Es el paso que responde al R6 (cuotas visibles antes de confirmar).
“Información de envíos” como sección propia	El link “Envíos” del menú lleva provisoriamente al carrito, donde está la calculadora de envío.	Hasta que se maquete la página informativa.

Agregados que no estaban en la propuesta

Agregado	Justificación UX
Botón flotante de WhatsApp en todas las páginas (no solo en la ficha de producto)	La proto-persona está acostumbrada a consultar por mensaje; tenerlo siempre a mano reduce la fricción.
Barra superior “Envío gratis desde $40.000 · Hasta 12 cuotas” y aviso en el carrito “Te faltan $X para el envío gratis”	Ataca la incertidumbre sobre el costo de envío desde la primera pantalla e incentiva un ticket mayor.
Opción “Retiro en punto Nutrite” (gratis) en la calculadora de envío	Alternativa sin costo que reduce el abandono por precio de envío.
Descuento por transferencia y tabla de cuotas con interés	Hace explícito el costo real de cada medio de pago antes de confirmar (R6).
Acordeón con información nutricional en la ficha	Retoma una necesidad que había surgido en el primer borrador de la proto-persona y que suma confianza al decidir.
Chips de filtros activos sobre la grilla y opción “Solo con stock”	Permite ver y quitar filtros rápido desde el celular, y ocultar productos no disponibles (HU1 + HU2).
Breadcrumb y paginación	Orientación dentro del catálogo; refuerza la estructura MPA indexable.

Las imágenes de productos son ilustraciones SVG propias usadas como marcador de posición; en producción se reemplazan por las fotos reales del catálogo (con el mismo tamaño cuadrado 1:1 que define el CSS). Precios, número de WhatsApp y dirección son datos de ejemplo.

Anexo — Uso de Inteligencia Artificial

Al igual que en el trabajo anterior, se utilizó Claude (Anthropic) como asistente para acelerar la maquetación y la redacción de este informe, a partir del documento de propuesta UX/UI y de las consignas del SP3.

Uso	Revisión realizada
Generación inicial del HTML y las hojas de estilo según el sitemap y los requerimientos funcionales.	Se verificó que cada requerimiento (R1–R6) tuviera su componente visible, que los nombres siguieran BEM de forma consistente y que no hubiera estilos en línea.
Pruebas de responsividad a 390, 820 y 1366 px.	Se detectó y corrigió un desborde horizontal en el carrito y la ficha de producto en celular (se resolvió con `minmax(0, 1fr)`), y se agregó un breakpoint de 768 px porque en tablet la grilla de 3 columnas dejaba una tarjeta huérfana.
Redacción del informe.	Se revisó que los cambios declarados coincidan con lo efectivamente desarrollado.